



Stages

Stage 1 (6 pts.)

Stage 2 (7 pts.)

Stage 3 (6 pts.)

Stage 4 (5 pts.)

Starting code

L8: Mages' Chess

A correspondence team of remote mages was bored during a telepath-conference. Archibald and Eleonora invented their own variant of chess - using familiars and fireballs! Until now, they used clairvoyance to play through the Net[1]. It worked well, but left the risk of seeing the enemy's future moves - giving an unfair advantage. Instead of casting spells blindly, they want to send datagrams.

[1] - The weave of magic permeating all material and spiritual reality.

Prepare a UDP server for them that reacts to messages of a fixed size of 16 bytes in the following format:

- 1 byte - a character (`char`) defining the message type
- 1 byte - no meaning, so-called *padding*
- 14 bytes - message body padded with null characters (`'\0'`), depending on its type:
 - `l` - `login` message - a string containing the player's name with a maximum length of 14
 - `c` - `cast` message - 3 two-byte integers (`uint16_t`) representing in turn the spell choice, the target X coordinate, and the target Y coordinate.
 - `q` - `quit` message - empty body

The program takes one argument, which is the port on which the server listens:

```
./sop-mages <port>
```

Stages

Stage 1 (6 pts.)

The program waits for datagrams on the given port. Accept 3 types of messages; after receiving the appropriate one, print to standard output:

- [Login] Welcome, <name>
- [Cast] Someone casts <spell_name> onto <X>,<Y>
- [Quit] Someone quit. Goodbye!

Additionally, the program ends after receiving and handling 4 messages.

After receiving a datagram of incorrect size, an incorrect first byte (other than 'l', 'c', or 'q'), an incorrect spell choice (outside the `spell_names` array), or incorrect coordinates (less than 0, greater than or equal to `BOARD_SIZE`), you should print an appropriate error message, but do not interrupt the server's operation.

Hint: When using a compiler from the gcc family, to ensure the structure has a predictable memory layout, you can use the `__attribute__((__packed__))` attribute - an example of use is in `sop-mag.c`.

Stage 2 (7 pts.)

After receiving a `cast` message, save the command with the received three numbers in a **FIFO** queue. In parallel with receiving datagrams, also run `THREAD_COUNT` familiar threads that pick up the oldest command and cast the spell - that is, wait `FAMILIAR_DELAY` ms and print an appropriate message to standard output as in stage 1. The queue has a capacity of `MAX_QUEUE` commands. When there is no room for another command, discard it, print an appropriate error message, and continue operation. Use a semaphore or a condition variable for synchronization.

Reminder: Active waiting (so-called busywaiting) is forbidden.

Stage 3 (6 pts.)

The program correctly handles logging in. Reject all messages except `login` before the start of the game - receiving a `login` message from two players. Players must have a different port and/or IP address. After the game starts, reject all messages not coming from players. Logged-in players start with 10 pebbles in their pouches - a certain measure of points in the game.

Spells cast by familiars will cost players pebbles. Depending on the spell type, it will be in turn: 1 pebble for "Divination" (spell choice 0), 3 for "Summon Elemental" (choice 1), and 4 for "Fireball" (choice 2). In case a player does not have enough pebbles, the familiar prints to standard output `[tee hee] Not enough pebbles, <name>!`, waits `FAMILIAR_DELAY` ms, and waits for the next command.

The message printed by the familiar for successfully casting a spell from this stage also refers to the name of the user casting it: `[Cast] <name> casts <spell_name> onto <X>, <Y>`.

A `quit` message from a logged-in player after the game starts means surrender. Their opponent then wins. Print a surrender message `[Quit] <name> quit. Goodbye!`, and then an announcement of the winner `-= Congratulations, <name>, you win! -=`. After that, the program ends.

Stage 4 (5 pts.)

The game takes place on a board with dimensions `BOARD_SIZE` x `BOARD_SIZE`. Each field can be empty or have an elemental belonging to one of the players on it.

In addition to the threads present so far, there is a judge thread. From the moment the game starts (i.e., both players login), every second it prints the status of the board and the players' pouches to standard output together with a legend explaining which character represents which player's elemental using their names.

Besides this, every second the judge changes the state of both players' pouches by $\Delta_K = \lfloor \frac{N}{2} \rfloor - 1$ pebbles, where N is the number of that player's elementals on the board. After such a change, the judge sends a 2-byte datagram with the current number of pebbles as an unsigned integer to each player. If a player has no more pebbles and $\Delta_K < 0$, their opponent wins. If this were to happen for both players simultaneously, the player who logged in first wins. The judge announces the winner on standard output `-= Congratulations, <name>, you win! -=` and sends a 1-byte datagram with the character `'w'` to the winner and `'l'` to the loser. After that, the program ends.

The "Divination" spell (spell choice 0) allows a player to know the contents of the fields in a 5x5 square around the indicated location. The server sends the player a datagram of 50 bytes containing 25 2-byte integers representing the contents of the 25 fields surrounding the spell target:

field content	empty	divining player's elemental	opponent's elemental	outside the board
sent number	0	1	2	3

A "Summon Elemental" type spell (spell choice: 1) summons an elemental belonging to the player casting the spell on a given field, if the selected field is empty. Otherwise, nothing happens, and the pebbles are wasted.

A "Fireball" type spell (spell choice: 2) destroys all (!) elementals in a 3x3 square around the indicated location.

After a successful spell cast, the familiar sends the player a 2-byte datagram containing the number of pebbles in that player's pouch. This means that after a divination spell, 2 datagrams are sent.

Starting code

- [Makefile](#)
- [l8_common.h](#)
- [sop-mag.c](#)

[Download as zip](#)